Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

FUSE: MusicFS

Виртуальная организация музыкальных файлов
по метаданным

Выполнил:
студент группы P3310
Чжун Цзяцзюнь

Проверил:
Максимов Андрей Николаевич

Санкт-Петербург, 2026

1. Цель и модель файловой системы

Цель работы - изучить обработку файловых запросов в пользовательском пространстве и реализовать MusicFS: файловую систему только для чтения, которая организует аудиофайлы по BPM, тональности и назначению. Постановка: FUSE в репозитории курса. Материалы реализации, запуска и проверки находятся рядом с этим отчётом; внешняя ссылка на репозиторий проекта не создавалась.

Входные данные - каталог с аудио и JSON-массив. Каждая запись задаёт уникальный `id`, относительный `path`, а также необязательные `bpm`, `key` и `type`. Метаданные загружаются при запуске. После их изменения требуется повторное монтирование.

```
{"id": "s001", "path": "drums/drum_loop.wav",
 "bpm": 124.5, "key": "A_minor", "type": "drum"}
```

Каталог	Правило
`by_bpm`	Ближайшее кратное 10; середина округляется вверх: 124,5 → 120; 125 → 130. Расчёт выполняется через Decimal.
`by_key`	Метка из JSON, например `A_minor`. Определение тональности по сигналу не выполняется.
`by_type`	Назначение: `drum`, `bass`, `vocal`, `song`. Расширения WAV/MP3 не смешиваются с назначением.
`unknown`	Группа для отсутствующего значения внутри соответствующей категории.

Виртуальное имя имеет вид `id__исходное_имя`. Это сохраняет расширение и различает одинаковые имена из разных исходных каталогов. Пустые категории не создаются. Абсолютные пути, выход за исходный корень, небезопасные имена и повторные записи отклоняются до монтирования. Недоступный исходный файл пропускается с предупреждением.

```
by_bpm/120/s001__drum_loop.wav
by_key/A_minor/s001__drum_loop.wav
by_type/drum/s001__drum_loop.wav
```

Три записи каталогов → один виртуальный inode → один исходный аудиофайл.

Физические каталоги категорий, копии аудио и жёсткие ссылки в исходной библиотеке не создаются. Идентификатор виртуального inode не обязан совпадать с inode исходной файловой системы.

2. Реализация и обработка запросов

Использованы Python, pyfuse3 и libfuse3. pyfuse3 предоставляет низкоуровневый интерфейс, работающий с inode; цикл запросов выполняется через Trio. Код разделён на три небольших модуля.

Модуль	Ответственность
`catalog.py`	Чтение и проверка JSON, таблица inode и отображения «имя → inode» внутри каталогов. Не зависит от FUSE.
`fs.py`	Обработчики запросов, атрибуты, исходные дескрипторы и чтение.
`__main__.py`	Параметры запуска, проверка точки монтирования, запуск и очистка.

Приложение (`ls`, `stat`, `open`, `read`)
↓ Linux VFS и интерфейс FUSE ядра
↓ libfuse3 / pyfuse3
↓ обработчики MusicFS
↓ индекс каталогов или чтение исходного файла

`lookup` находит inode по имени в родительском каталоге; `getattr` возвращает тип, права, размер и время. Несколько записей каталогов содержат один inode аудио, поэтому число виртуальных ссылок равно трём. `opendir` использует inode каталога как его дескриптор. `readdir` передаёт записи со стабильными позициями и прекращает отправку при заполнении буфера ядра.

`open` проверяет режим и открывает исходный файл с `O_RDONLY`. Полученный файловый дескриптор сохраняется как `fh`. `read` вызывает:

```
data = await trio.to_thread.run_sync(os.pread, fh, size, off)
```

`pread` читает заданный диапазон без изменения общей позиции дескриптора. Блокирующее чтение выполняется рабочим потоком. На EOF возвращается сокращённый результат или пустой массив байтов. `release` закрывает дескриптор, общий обработчик завершения закрывает оставшиеся ресурсы.

Для защиты от изменений используются `ro`, `default_permissions`, права `0555` для каталогов и `0444` для файлов. `open` дополнительно запрещает режимы записи. Ошибки ОС преобразуются в `FUSEError` с соответствующим errno. Индекс неизменяем до размонтирования, поэтому inode не перераспределяются и в `forget` не освобождаются.

3. Запуск и методика проверки

Проверка выполнена в WSL2, Ubuntu 22.04.5 LTS: Python 3.10.12, libfuse3 3.10.5, pyfuse3 3.5.0, Trio 0.35.0. Полные версии сохранены в `evidence/environment.txt` и `evidence/python-packages.txt`.

После установки зависимостей и активации отдельного виртуального окружения воспроизводимый запуск из корня проекта имеет вид:

```
python -m pip install -e .
python tools/make_demo.py demo
mkdir -p ~/musicfs-mnt
python -m musicfs --metadata demo/metadata.json \
  --source-root demo/audio ~/musicfs-mnt --debug
```

Генератор отказывается перезаписывать непустой каталог. В поставке `demo` уже создан; повторная генерация должна использовать другой пустой каталог. Для просмотра применяется второй терминал. Размонтирование: `fusermount3 -u ~/musicfs-mnt` или Ctrl+C в процессе MusicFS.

Для тестов используется стандартный `unittest`. Без переменной окружения реальные монтирования явно пропускаются; полный запуск:

```
MUSICFS_INTEGRATION=1 python -m unittest discover -s tests -v
```

Проверки	Содержание	Результат
11	JSON, BPM, категории, одинаковые имена, дубликаты, ссылки за корень, генератор WAV	Пройдены
8	Атрибуты, диапазоны чтения, EOF, режимы открытия, позиции каталога, удалённый источник, очистка, точка монтирования	Пройдены
11	Реальные монтирования FUSE: чтение, inode, запрет изменений, Unicode, снимок метаданных, каталог из 1200 элементов и завершение процесса	Пройдены
Всего: 30	Полный вывод: `evidence/tests.log`	**OK, без пропусков**

Использованы три синтетических PCM WAV длительностью одна секунда. Для каталога из 1200 элементов созданы отдельные короткие байтовые фикстуры: это проверка обхода каталога, а не воспроизведения аудио. Чтение проверено в начале, середине, на границе и после EOF; 50 диапазонных чтений выполнены с восемью рабочими потоками.

Интеграционные тесты работают только с временными исходными каталогами и точками монтирования в Linux. Они не изменяют личную музыкальную библиотеку. Тесты завершают процессы и проверяют размонтирование перед удалением временных данных.

4. Полученные результаты и исправленная ошибка

В отдельном демонстрационном монтировании три классификационных пути для `s001__drum_loop.wav` дали следующие реальные атрибуты:

Путь категории	inode	Ссылки	Байты
`by_bpm/120`	2	3	44144
`by_key/A_minor`	2	3	44144
`by_type/drum`	2	3	44144

SHA-256 содержимого всех трёх входов совпал с SHA-256 исходного файла:

`d0cb6df7ee404992aa37fb5cc150c586d592e4bf55e4ec05056ec150f41f1a8e`

Создание файла в точке монтирования завершилось `EROFS`, errno 30. Хеш исходного файла после этой попытки остался тем же. В полном наборе также отклонены запись, перезапись, удаление, переименование, усечение, смена прав и создание каталога.

Системные команды `ls`, `stat` и `cp` успешно обращались к монтированию. Каталог с 1200 файлами был прочитан без пропусков и повторов. Изменение JSON не меняло текущую структуру, а после повторного монтирования новая категория стала доступна. SIGINT, SIGTERM и искусственно вызванное исключение приводили к очистке; повторный запуск проходил успешно.

4.1. Ошибка, найденная реальным монтированием

Первый полный прогон выявил ошибку чтения ранее открытого файла после удаления исходного имени. В обработчике `read` уже использовался открытый fd, но ядро перед чтением повторно запрашивало атрибуты. `getattr` выполнял `stat` по исчезнувшему пути и возвращал `ENOENT`.

Исправление: если исходный путь исчез, но для inode ещё открыт дескриптор, `getattr` получает атрибуты через `fstat(fd)`. Новое открытие по-прежнему обращается к исходному пути и завершается `ENOENT`. После закрытия последнего дескриптора получение атрибутов также возвращает ошибку. Повторный полный прогон завершился успешно.

Подробности демонстрации сохранены в `evidence/demo.txt`, `summary.json` и `mount.log`. Тест подтверждает семантику дескрипторов Linux; удаление через сам MusicFS остаётся запрещённым.

5. Ограничения, работа с LLM и вывод

5.1. Границы реализации

Метаданные и индекс фиксируются при запуске. Горячая перезагрузка, изменение тегов и запись не реализованы. Изменение содержимого исходного аудио во время монтирования не поддерживается; после изменения библиотеки требуется повторный запуск. Безопасная работа при одновременной злонамеренной замене исходных каталогов не заявляется.

Система читает исходные байты независимо от расширения. Реальные MP3/FLAC, проигрыватели, распознавание BPM и тональности, а также производительность большой аудиобиблиотеки не проверялись. Успешное чтение и совпадение хешей не являются проверкой декодирования или воспроизведения. SIGKILL не позволяет выполнить Python-очистку и может требовать ручного размонтирования.

5.2. Методика работы с LLM

LLM использовалась для уточнения требований, подготовки Python-кода, тестов, документации и черновика отчёта. В запросах были зафиксированы три модуля, JSON, режим только чтения, правило BPM и требование реального монтирования. Использованы официальные описания libfuse/pyfuse3; результат проверялся исполнением кода и тестов.

После первого прогона в запрос на исправление были включены конкретная ошибка `ENOENT`, последовательность «открыть - удалить источник - прочитать» и необходимость сохранить поведение новых открытий. Затем выполнен полный повторный прогон. Внешние сервисы в проекте отсутствуют, поэтому отдельного сгенерированного модуля взаимодействия с ними нет. Для защиты требуется самостоятельное понимание и объяснение кода; автоматическое выполнение тестов не заменяет проверку преподавателем.

5.3. Вывод

Реализован MusicFS с виртуальной организацией аудиофайлов по трём признакам. Показаны связь каталогов и inode, обработка атрибутов и диапазонного чтения, защита от записи и очистка ресурсов. Все 30 автоматических тестов, включая 11 проверок с настоящим FUSE, прошли. Результаты относятся к указанным фикстурам и сценариям; приёмка преподавателем не заявляется.

5.4. Источники

1. Курс ОС. Лабораторная работа FUSE.
2. libfuse. Официальная документация.
3. pyfuse3. Общие сведения и жизненный цикл.
4. pyfuse3. Обработчики запросов.
5. pyfuse3. Примеры файловых систем.
6. Курс ОС. Правила составления отчёта.